



TOURISE 2027 — Partner API

Request an Invitation & Newsletter · updated 1 October 2026

Two endpoints behind one key: file an **invitation request**, and **subscribe** someone to the newsletter. Your website hosts its own forms and sends the results to TOURISE. A request filed this way lands in the same review queue as one filed on the TOURISE site, under the category you were given.

It is a single JSON `POST`. There is no SDK to install and nothing to embed.

Three things TOURISE gives you — and one thing you give us

API base URL `https://<api-host>/api`

Category slug e.g. `press`

API key `tri_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX`

You give us: the public IP address(es) of the server that will call the API — single addresses or a range such as `203.0.113.0/24`. The key works only from those addresses.

The key is shown once when it is issued. If it is lost, TOURISE issues a new one — that is also how it is rotated.

How it works

Server to server only. Your page posts to your own server; your server adds the key and calls TOURISE. The key never reaches a browser.

Every call is checked twice KEY + SERVER IP

- **The key** says which partner you are.
- **The server IP** says the call comes from your server. It must be one of the addresses you gave TOURISE, so your server needs a **fixed public outgoing IP** — a VPS or cloud server, or serverless behind a static egress IP. Plain Vercel, Netlify or Cloudflare Workers functions change address and will be refused.

Step 1 — Check your key, from your server

GET `/api/partner/verify` · key required · creates nothing

The one call that proves your set-up end to end: the key, and the server it is sent from. Run it from the server that will file requests, as often as you like — nothing is created and nothing reaches the review queue.

```
GET /api/partner/verify
X-API-Key: tri_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

// 200 → { "status": "success", "data": { "valid": true, "site": "Your site name", ... } }
// 401 → the key is missing, unknown or revoked
// 403 → not allowed: see "Getting 403?" below
```

A 200 FROM THE FIELD LIST DOES NOT TEST YOUR KEY

Step 2 is public and never reads the key, so it answers 200 whatever you send. Only this call, or a real **POST**, tells you the key and your server are accepted.

Step 2 — Read the field list

GET `/api/request-an-invitation/{slug}` · public, no key needed

Tells you which fields to render and which to require, so your form follows the category without a redeploy when TOURISE changes it.

```
{
  "status": "success",
  "data": {
    "slug": "press",
    "name_en": "Press",
    "with_email_otp": true,
    "mandatory_fields": ["first_name", "company"]
  }
}
```

`null` means the category is not configured — nothing beyond `email` is required. `with_email_otp` tells you which flow to build: `false` → file straight away; `true` → TOURISE emails the applicant a code first. Both are in *The two flows* below.

The two flows

Which one you build depends on one value from Step 2. Read it from the field list, not from this page: TOURISE can switch a category between the two.

STEP 2 SAYS	BUILD	CALLS TO TOURISE
"with_email_otp": false	Flow 1 — without a code	One: file the request.
"with_email_otp": true	Flow 2 — with a code	Two: send the code, then file the request with it.

Step 0 is optional in both: it only makes the answer arrive sooner, since filing and `send-code` run the same check anyway. In the charts, each row is one step and the coloured cell is who acts: **the applicant** on your page, **your server**, or **TOURISE**. Your page never talks to TOURISE directly.

Flow 1 — without a code

#	APPLICANT · YOUR PAGE	YOUR SERVER	TOURISE
0	Types their email. As they leave the field, your page asks your server.	Checks it: <code>POST /request-an-invitation/{slug}/check-email</code> . On <code>409</code> , your page shows “already registered” under the field right away.	Answers <code>200</code> or <code>409</code> . Creates and sends nothing.
1	Fills in the rest of your form and presses Submit . → to your server		
2		Adds the key and files the request: <code>POST /request-an-invitation/{slug}</code> → to TOURISE	
3			Checks the key and your server's IP, then the fields, then whether the address is already on file. Files it and answers <code>201</code> . ← back to your server
4	Sees your thank-you. done	Passes the result to your page.	

YOUR FORM

The fields from Step 2, and **Submit**.

AFTER SUBMIT

`201` → thank-you.

`409` → “This email is already registered / has already requested an invitation.”

`422` → the field errors.

Flow 2 — with a code

#	APPLICANT · YOUR PAGE	YOUR SERVER	TOURISE
0	Types their email. As they leave the field, your page asks your server.	Checks it: <code>POST /request-an-invitation/{slug}/check-email</code> . On <code>409</code> , your page shows “already registered” under the field right away.	Answers <code>200</code> or <code>409</code> . Creates and sends nothing.
1	Presses Send code . → to your server		
2		Asks TOURISE to send the code: <code>POST /request-an-invitation/{slug}/send-code</code> <code>{ "email": "...", "lang": "en" }</code> → to TOURISE	
3			Checks the key and your server's IP. Emails the applicant an 8-character code and answers <code>200</code> — without the code. → email to the applicant · ← <code>200</code> to your server
3b		Instead of 3, gets <code>409</code> : the address is already on file and no email was sent . Show the message and stop here.	
4	Opens the email, types the code into your form with the other fields, presses Submit . → to your server		
5		Files the request with the code: <code>POST /request-an-invitation/{slug}</code> fields + <code>"otp_token": "A1B2C3D4"</code> → to TOURISE	
6			Checks the code against that email, files the request, spends the code. Answers <code>201</code> . ← back to your server
6b		Instead of 6, gets <code>422</code> with <code>reason: otp_invalid</code> : the code is wrong, or a newer one replaced it. Let the applicant try again or press Send code again (back to 1).	
7	Sees your thank-you. done	Passes the result to your page.	

SCREEN 1

Email field and a **Send code** button.

SCREEN 2

"We sent a code to ...", a code box, the other fields from Step 2, **Submit**, and a *Send a new code* link.

AFTER SUBMIT

201 → thank-you.
422 otp_invalid → "That code is not right — check it or send a new one."

THE CODE IS BETWEEN TOURISE AND THE APPLICANT

It goes from us straight to their inbox, and the only copy your server ever handles is the one the applicant types into your form. Never in the `send-code` response, never in a log of ours you can read.

Check the email as they type

POST `/api/request-an-invitation/{slug}/check-email`

· key required · creates nothing

```
POST /api/request-an-invitation/press/check-email
X-API-Key: tri_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
Content-Type: application/json

{ "email": "sara@example.com" }

// 200 → { "status": "success", "data": { "available": true } }
// 409 → { "status": "failed", "reason": "already_registered" | "already_requested", "message":
"..." }
```

- Call it when the applicant **leaves the email field**, or after they stop typing for about half a second — not on every key press. It shares the 60-a-minute limit with your other calls.
- It is a hint for the form, not a reservation: always handle `409` on filing too.

Send the code

POST `/api/request-an-invitation/{slug}/send-code`

· key required

```
POST /api/request-an-invitation/press/send-code
X-API-Key: tri_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
Content-Type: application/json

{ "email": "sara@example.com", "lang": "en" }

// 200 → { "status": "success", "data": { "otp_required": true } }   code emailed
// 409 → already on file (reason says which) — no email sent
// 401 / 403 / 404 / 410 / 422 / 429 → as in Step 3
```

- `lang` (`en` or `ar`) picks the language of the email. Defaults to `en`.
- The code is **never** in the response. Only the applicant sees it.
- One code is live per address: asking again replaces the previous one, which then stops working.

- The code works only for the address it was sent to, is not case-sensitive, and is spent when the request is filed.

Step 3 — File the request

POST `/api/request-an-invitation/{slug}` · key required

```
POST /api/request-an-invitation/press
X-API-Key: tri_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
Content-Type: application/json

{
  "email": "sara@example.com",
  "first_name": "Sara",
  "last_name": "Idris",
  "company": "Partner Co",
  "job_title": "Editor",
  "otp_token": "A1B2C3D4" // only when with_email_otp is true
}
```

Authorization: Bearer <key> is accepted as the same credential if that header is easier for your HTTP client.

SUCCESS

```
HTTP 201 { "status": "success", "data": { "id": "01a0...", "duplicate": false } }
```

email is always required. Everything else is required only if the category says so, and the list in Step 2 is the live answer — it follows the category without a redeploy. Any field not on that list is ignored, apart from otp_token .

COUNTRIES ARE UIDS

country_of_residence and nationality take the id from GET /api/countries — not "Saudi Arabia" . Fetch that list once and use it to populate your dropdowns.

Responses

CODE	MEANING · WHAT TO DO
201	Filed. Show your thank-you.
401	Key missing, unknown or revoked. Not retryable — contact TOURISE.
403	Not allowed. Not retryable as is — see <i>Getting 403?</i> below.
404	No category at that slug. Check your spelling of the slug.
409	That address is already on file — nothing new created. reason is already_registered (already a guest) or already_requested (an earlier request, whatever its outcome). Tell the applicant; do not retry.

410	The category exists but is closed. Stop showing the form and say applications have closed.
422	Validation failed. <code>errors</code> names each field — show them against your inputs. With <code>reason: otp_invalid</code> : the code is missing, wrong or was replaced — let the applicant ask for a new one.
429	Rate limited (60/min per key). Back off and retry.

409 IS A MESSAGE, NOT A FAULT

Every address is checked against the guest list and every earlier request, so one person is never in the queue twice. Show the applicant why, and do not retry. `duplicate` in the 201 body is always `false`.

Email verification (OTP)

Turned on per category by TOURISE, and reported as `with_email_otp` in Step 2. When it is on, a request filed without a valid code is refused — follow *With a code* in *The two flows*.

WHY THE CODE COMES FROM US

TOURISE sends it and checks it, so the applicant proves they own the address without your server ever holding the proof. The invitation, if the request is accepted, goes to that same address.

After the request is filed

Who talks to the applicant next, and what each side keeps.

What TOURISE sends them

MOMENT	WHAT WE SEND	WHEN NOTHING GOES OUT
Code requested (send-code)	The 8-character code, from TOURISE.	Never when the address is already on file (409).
Filed (201)	A holding email — it arrived and is waiting on a decision.	Only if the category names a template for it. Otherwise the applicant hears nothing yet.
Already on file (409)	Nothing. The earlier record still stands.	Always — a repeat submit sends no email.
Accepted	Their invitation, with the link to register, on the channel that category uses.	—
Rejected	A rejection email.	Only if the category names a template. Otherwise the request is closed silently.

ASK BEFORE YOU WORD YOUR THANK-YOU

Whether the holding email exists is a setting on your category, not a promise of the API. Ask TOURISE which of the moments above are switched on for you. If there is no holding email, do not write "check your inbox" — say the request was received and they will hear when it is reviewed.

What TOURISE keeps

The request row: the fields you sent, the language, which partner site filed it, its status, and the reviewer's note. A newsletter opt-in is kept separately as a subscriber, attributed to your site the same way. Nothing else about the visitor reaches us — we see no page views, no cookies and no visitor IP addresses. The only address we record is your server's, on your key, to show which address your last accepted call came from.

What you keep

Whatever you choose: the API needs no copy on your side, and the review queue is the record TOURISE works from. If you do keep one, say so in your own privacy notice — it is your form and your visitor. An applicant asking to be removed from the TOURISE queue should be passed to your TOURISE contact; you cannot delete through this API.

Example — your server-side endpoints

Two small endpoints on your server. The second is all you need for a category without a code.

```
// POST /api/request-invitation/send-code — only for a category with a code.
export async function sendCode(req, res) {
  const upstream = await fetch(
    `${process.env.TOURISE_API_BASE}/request-an-invitation/press/send-code`,
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json',
        'X-API-Key': process.env.TOURISE_API_KEY,
      },
      body: JSON.stringify({ email: req.body.email, lang: req.body.lang }),
    }
  );
  // 200 = code emailed. 409 = already on file (body.reason). Pass it on.
  res.status(upstream.status).json(await upstream.json());
}
```

```
// POST /api/request-invitation – on your server (fixed outgoing IP).
// Forward otp_token with the rest of the form when the category has a code.
// The key stays here and never reaches the browser.
export default async function handler(req, res) {
  const upstream = await fetch(
    `${process.env.TOURISE_API_BASE}/request-an-invitation/press`,
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json',
        'X-API-Key': process.env.TOURISE_API_KEY, // server-side only
      },
      body: JSON.stringify(req.body),
    }
  );

  // 201 = created. 409 = already a guest, or already requested (body.reason
  // says which). 422 = field errors. 410 = closed. Pass them on to your page.
  const body = await upstream.json();
  res.status(upstream.status).json(body);
}
```

Server-to-server only

The key works only from your server, and only from the server IP addresses you gave TOURISE. A call made with it from a browser, or from any other address, is refused with `403`. If your server's outgoing address changes, send TOURISE the new one before you switch. Your page posts to your own server; your server adds the key and calls TOURISE. There is no CORS set-up to ask for.

Any server language works — Node, PHP, Python, Java, .NET or plain `curl`. No special headers are needed beyond the ones in the examples.

Getting 403?

The answer is a bare `Not allowed.` on purpose. Check these, in order:

1. **The call leaves from your server**, not from a page, a browser extension or a tool running in the browser.
2. **It leaves from an address TOURISE has on your key.** The address that counts is the one the internet sees your server call from, which can differ from the machine's own address — behind a NAT gateway, a proxy, or over IPv6. If unsure, ask TOURISE which address your last call arrived from.
3. **You build the headers yourself.** Do not forward the headers of the request your page sent you (`headers: req.headers`) — send only `X-API-Key`, `Content-Type` and `Accept`, as in the example above.

Then run `GET /api/partner/verify` again: a `200` there means filing will be accepted too.

Before you go live

1. Send TOURISE your server's public outgoing IP address(es), and wait for confirmation they are on your key.
2. Call `GET /api/partner/verify` **from that server** and confirm you get `200`.

3. GET the field list and confirm your form asks for everything in `mandatory_fields` .
4. Submit once and confirm you get `201` .
5. Type an address that already requested into your form and confirm `check-email` shows the message before submit.
6. Submit the same address again and confirm you get `409` with `reason: already_requested` — and that your UI tells the applicant, rather than showing a generic error.
7. Submit with a missing required field and confirm you render the `422` errors against the right inputs.
8. Submit with a deliberately wrong key and confirm you get `401` and show a sensible message.
9. If `with_email_otp` is `true` : call `send-code` , confirm the email arrives, file with the code and get `201` . Then file with a wrong code and confirm you show the `otp_invalid` message.
10. Ask TOURISE to confirm the request appeared in the review queue.
11. Confirm the key lives only in your server's environment, never in anything shipped to the browser.

Good to know

- Submitting is a **request to be reviewed** — not a registration and not a confirmed place. Say so on your form, or applicants will expect a ticket.
- TOURISE records which partner site filed each request, so your submissions are attributed to you automatically. You do not send that.
- The rate limit is per key, so another partner cannot use up your budget.
- Nothing here needs Google reCAPTCHA or any browser set-up such as CORS.

Newsletter signup

The same key, for a signup box on your site.

POST /api/newsletter/subscribe

```
POST /api/newsletter/subscribe
X-API-Key: tri_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
Content-Type: application/json

{
  "email": "sara@example.com",
  "first_name": "Sara",
  "last_name": "Idris",
  "company": "Partner Co",
  "lang": "en"
}
```

Only `email` is required. New subscribers join the default list automatically. Like filing a request, it is called from your server, with your key, from an address on your key.

Say the right thing back

Success is always `201`. Read `state` — "Thanks for subscribing" shown to someone who signed up months ago reads as though the first time never took.

STATE	MEANING	SAY
subscribed	New — now on the list	"Thanks for subscribing"
already_subscribed	Was already on the list	"You are already subscribed"
resubscribed	Had unsubscribed, now back	"Welcome back"

`401` for a missing, unknown or revoked key. `403` when a call with the key is not allowed — the same rules as filing a request: from your server, from an address on your key. `422` if the address is not a valid email.

ABOUT THE TOKEN FIELD

`token` comes back only when `state` is `subscribed` — a genuinely new address. It is a bearer credential: whoever holds it can read that subscriber and unsubscribe them, so it is never returned for an address that already existed, where the caller may know the address without owning it. Do not build a flow that needs it back for an existing subscriber.

ATTRIBUTION

TOURISE records which partner site each subscriber came from, taken from your key rather than anything you send. The first site to subscribe an address keeps the attribution — a later signup elsewhere does not rewrite where they originally came from.

TOURISE 2027 · Partner API · Request an Invitation and Newsletter · integration guide.

Questions, a new key, or a change of server address — contact your TOURISE technical contact.